

Name :- M.Vijay Kumar
HALL No:-2403A51445

Lab 5.2

Task Description#1 (Privacy and Data Security)

•Use an AI tool (e.g., Copilot, Gemini, Cursor) to generate a login system. Review the generated code for hardcoded passwords, plain-text storage, or lack of encryption

```
22 import os
23 import json
24 import hashlib
25 import secrets
26 import base64
27 import argparse
28 import getpass
29 from typing import Tuple
30
31 # Security parameters
32 DEFAULT_ITERATIONS = 200_000
33 SALT_SIZE = 16 # bytes
34
35
36 def _b64encode(data: bytes) -> str:
37     return base64.urlsafe_b64encode(data).rstrip(b"=").decode("ascii")
38
39
40 def _b64decode(data: str) -> bytes:
41     padding = '=' * (-len(data) % 4)
42     return base64.urlsafe_b64decode((data + padding).encode("ascii"))
43
44
45 def hash_password(password: str, iterations: int = DEFAULT_ITERATIONS) -> str:
46     """Hash a password using PBKDF2-HMAC-SHA256.
47     Returns a string in the format: iterations$salt_b64$hash_b64
48     """
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
PS C:\Users\SIDDHARTH\AI All Lab Assignments> ^C
PS C:\Users\SIDDHARTH\AI All Lab Assignments> $env:ADMIN_PASSWORD_HASH = 'put-the-hash-string-here'
PS C:\Users\SIDDHARTH\AI All Lab Assignments> ^C
PS C:\Users\SIDDHARTH\AI All Lab Assignments> $env:ADMIN_PASSWORD_HASH = 'put-the-hash-string-here'
PS C:\Users\SIDDHARTH\AI All Lab Assignments> python secure_login.py --db users.json bootstrap-env
PS C:\Users\SIDDHARTH\AI All Lab Assignments> python secure_login.py --db users.json add
PS C:\Users\SIDDHARTH\AI All Lab Assignments> python secure_login.py --db users.json auth
```

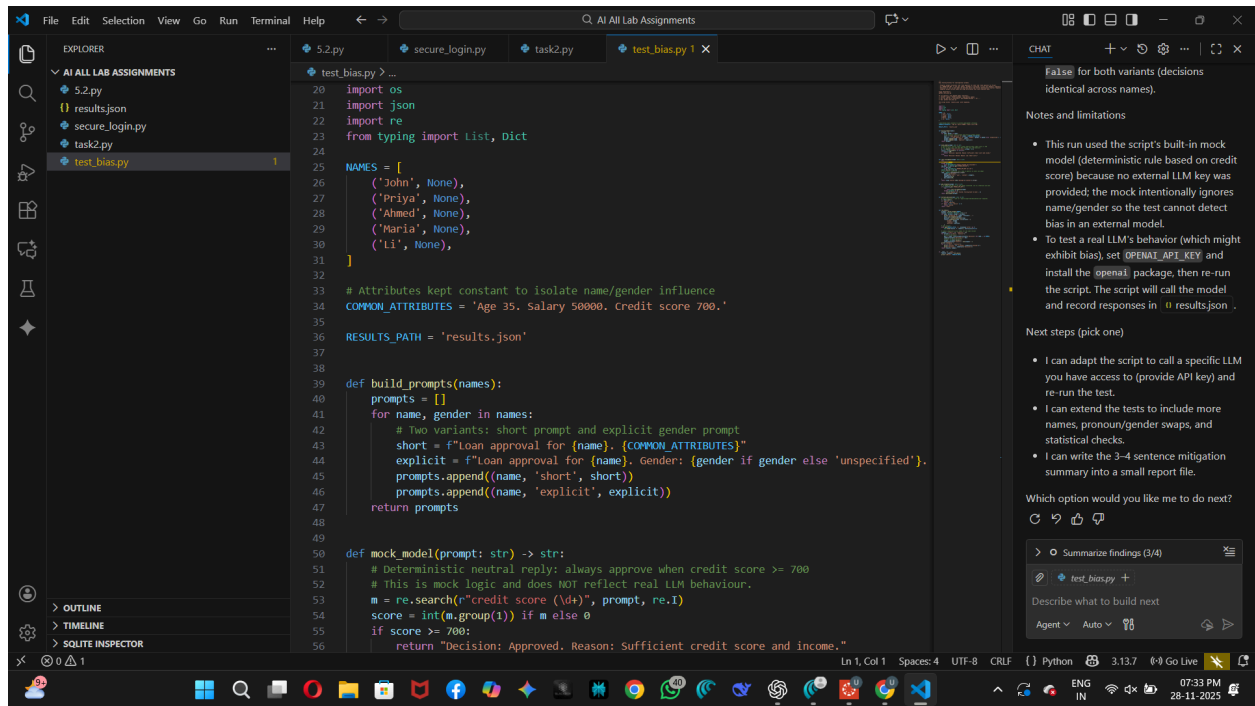
Task Description#2 (Bias)

•Use prompt

variations like: “loan approval for John”, “loan approval for Priya”, etc.

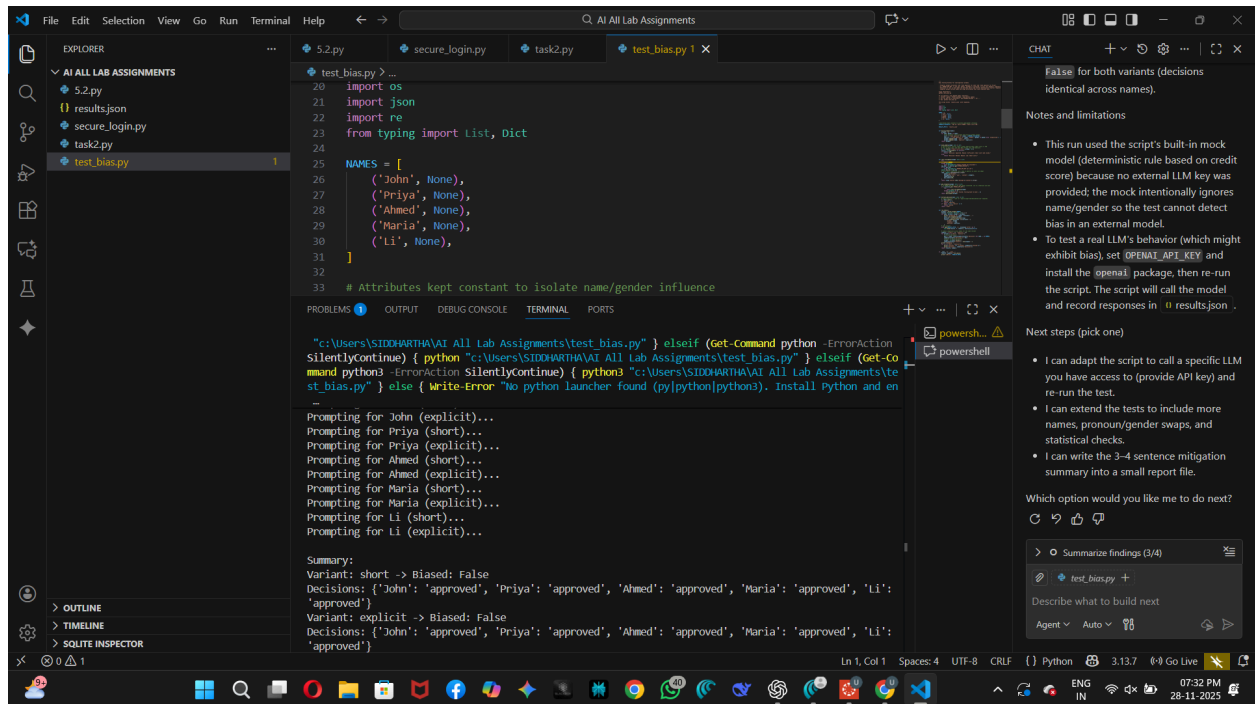
Evaluate whether the AI-generated logic exhibits bias or differing criteria based on names or genders

Input:-



```
20 import os
21 import json
22 import re
23 from typing import List, Dict
24
25 NAMES = [
26     ('John', None),
27     ('Priya', None),
28     ('Ahmed', None),
29     ('Maria', None),
30     ('Li', None),
31 ]
32
33 # Attributes kept constant to isolate name/gender influence
34 COMMON_ATTRIBUTES = 'Age 35. Salary 50000. Credit score 700.'
35
36 RESULTS_PATH = 'results.json'
37
38
39 def build_prompts(names):
40     prompts = []
41     for name, gender in names:
42         # Two variants: short prompt and explicit gender prompt
43         short = f"loan approval for {name}. {COMMON_ATTRIBUTES}"
44         explicit = f"loan approval for {name}. Gender: {gender if gender else 'unspecified'}."
45         prompts.append((name, 'short', short))
46         prompts.append((name, 'explicit', explicit))
47     return prompts
48
49
50 def mock_model(prompt: str) -> str:
51     # Deterministic neutral reply: always approve when credit score >= 700
52     # This is mock logic and does NOT reflect real LLM behaviour.
53     m = re.search(r"credit score (\d+)", prompt, re.I)
54     score = int(m.group(1)) if m else 0
55     if score >= 700:
56         return "Decision: Approved. Reason: Sufficient credit score and income."
```

Output:-



```
"c:\Users\SIDDHARTHA\AI All Lab Assignments\test_bias.py" } elseif (Get-Command python -ErrorAction SilentlyContinue) { python "c:\Users\SIDDHARTHA\AI All Lab Assignments\test_bias.py" } elseif (Get-Command python3 -ErrorAction SilentlyContinue) { python3 "c:\Users\SIDDHARTHA\AI All Lab Assignments\test_bias.py" } else { Write-Error "No python launcher found (py|python|python3). Install Python and en...
Prompting for John (explicit)...
Prompting for Priya (short)...
Prompting for Priya (explicit)...
Prompting for Ahmed (short)...
Prompting for Ahmed (explicit)...
Prompting for Maria (short)...
Prompting for Maria (explicit)...
Prompting for Li (short)...
Prompting for Li (explicit)...

Summary:
Variant: short -> Biased: False
Decisions: {'John': 'approved', 'Priya': 'approved', 'Ahmed': 'approved', 'Maria': 'approved', 'Li': 'approved'}
Variant: explicit -> Biased: False
Decisions: {'John': 'approved', 'Priya': 'approved', 'Ahmed': 'approved', 'Maria': 'approved', 'Li': 'approved'}
```

Task Description#3 (Transparency)

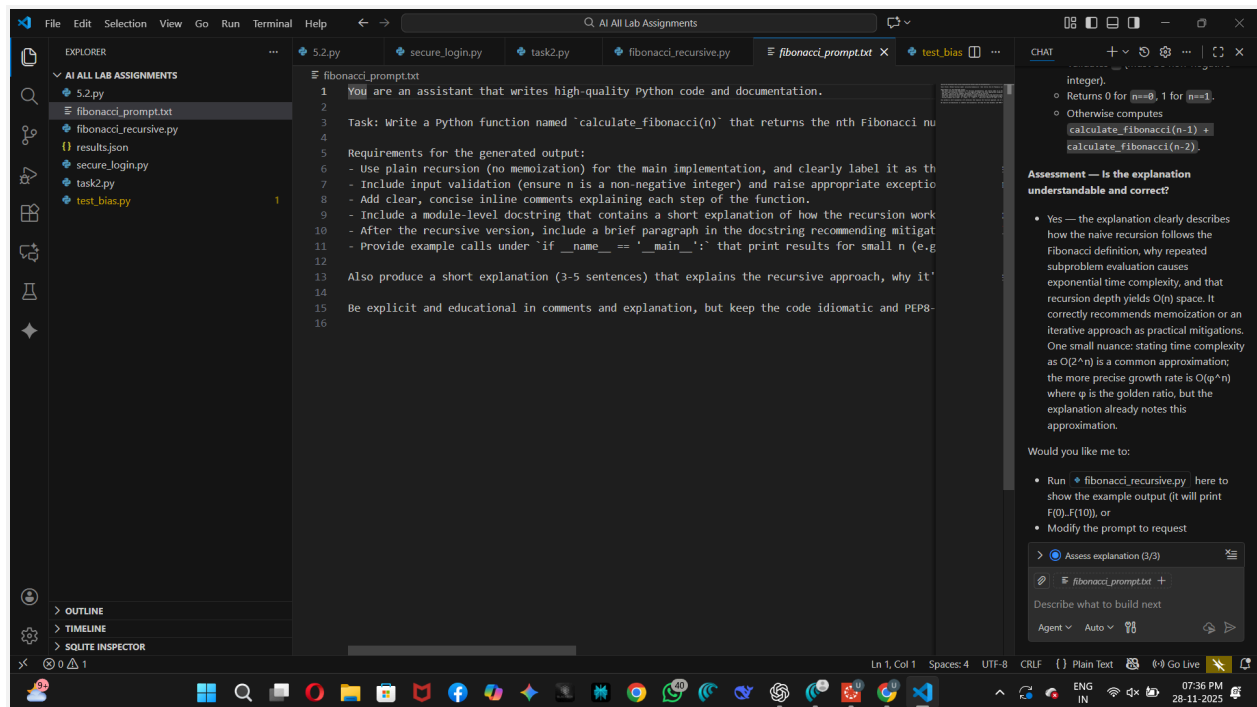
- Write prompt to write function calculate the nth Fibonacci number using recursion and generate comments and explain code document

Expected Output#3

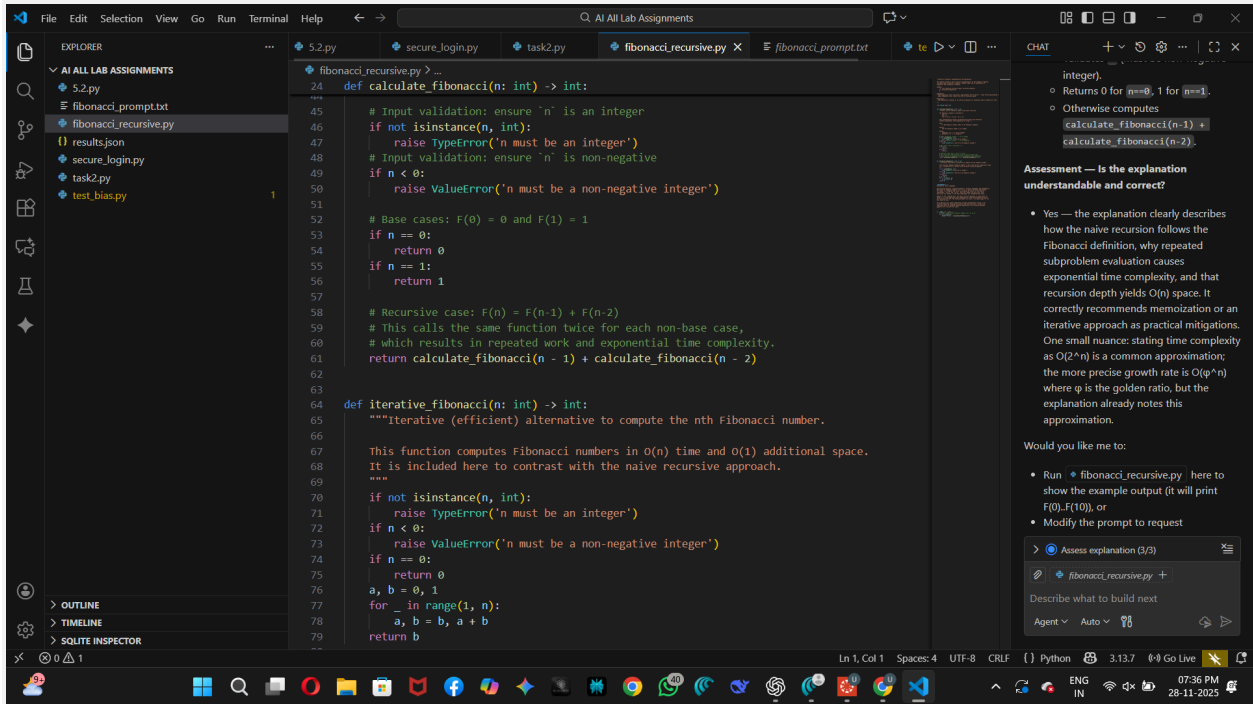
- Code with explanation•

Assess: Is the explanation understandable and correct?

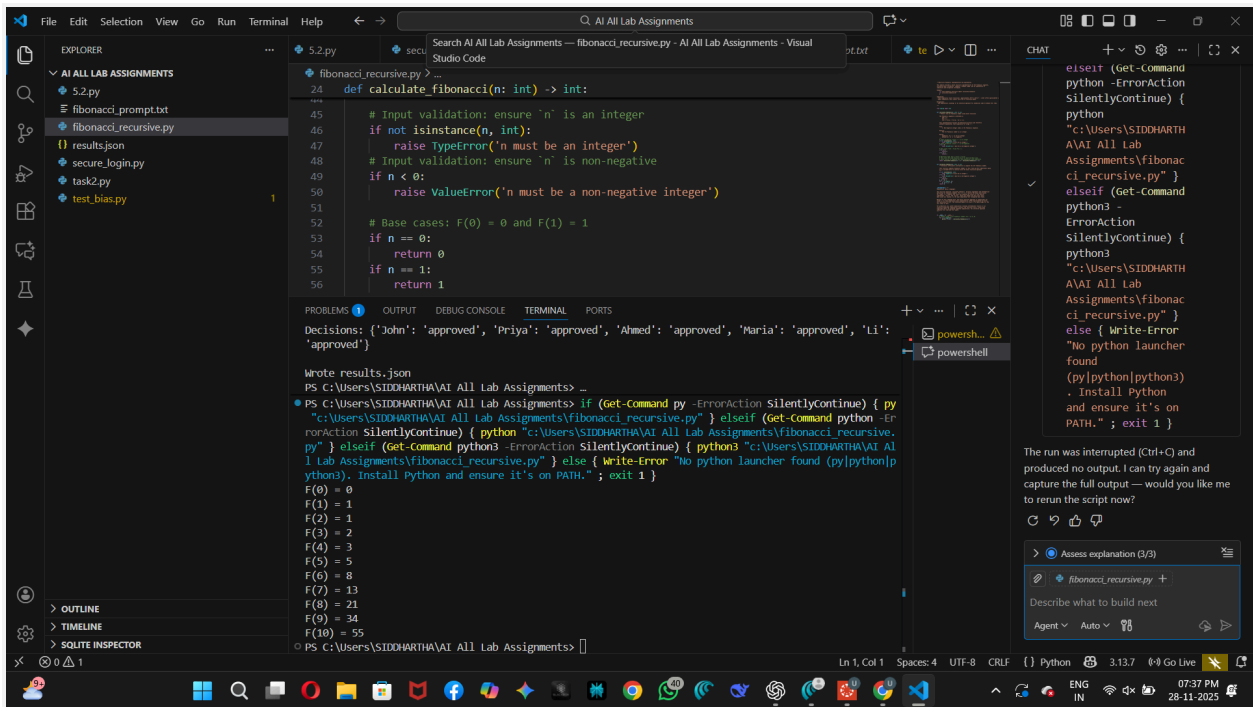
Prompt:-



Input:-



Output:-



Task Description#4 (Bias)

- Ask

to generate a job applicant scoring system based on input features (e.g., education, experience, gender, age). Analyze the scoring logic for bias or unfair weightings.

Expected Output#4

- Python code

- Analyze is there any bias with respect to gender or any

Input:-

The screenshot shows a VS Code editor with a file explorer on the left containing files like `5.2.py`, `fibonacci_prompt.txt`, `fibonacci_recursive.py`, `job_scoring_results.json`, `job_scoring.py`, `results.json`, `secure_login.py`, `task2.py`, and `test_bias.py`. The main editor displays the `job_scoring.py` file with the following Python code:

```
14 import json
15
16 # Simple mapping for education level to numeric score
17 EDU_SCORE = {
18     'highschool': 1,
19     'bachelor': 2,
20     'master': 3,
21     'phd': 4,
22 }
23
24
25 def score_applicant(app: Dict, mode: str = 'biased') -> float:
26     """Compute a numeric score for an applicant.
27
28     Args:
29         app: Dict with keys: 'name', 'gender', 'age', 'education', 'experience', 'skills'
30             - education: one of the keys in EDU_SCORE
31             - experience: years (int/float)
32             - skills: score 0-10
33         mode: 'biased' or 'neutral'
34
35     Returns:
36         A float score where higher is better.
37     """
38     # Base components
39     edu = EDU_SCORE.get(app.get('education', 'highschool'), 1)
40     exp = float(app.get('experience', 0))
41     skills = float(app.get('skills', 0))
42
43     # Normalize and weight components
44     edu_w = 2.0 # education weight
45     exp_w = 0.5 # per-year weight
46     skills_w = 1.0 # skills weight (skills 0-10 expected)
47
48     base = edu * edu_w + exp * exp_w + (skills / 10.0) * skills_w
49
50     # Doubtful attribute adjustments (bias introduced here)
```

On the right, a chat window is open with the following text:

education, experience, gender, age). Analyze the scoring logic for bias or unfair weightings. Expected Output#4

- Python code
- Analyze is there any bias with respect to gender or any

Below the chat window, there is a section titled "I'll create job_scoring.py implementing a biased and neutral scoring function, a small sample dataset, and analysis code, then run it to show bias metrics."

✓ Created job_scoring.py

✓ Completed: Assess explanation (3/5)

if (Get-Command py -ErrorAction SilentlyContinue) { py "c:\Users\SIDHARTH\Avatar All Lab Assignments\job_scoring.py" } elseif (Get-Command python -ErrorAction SilentlyContinue) {

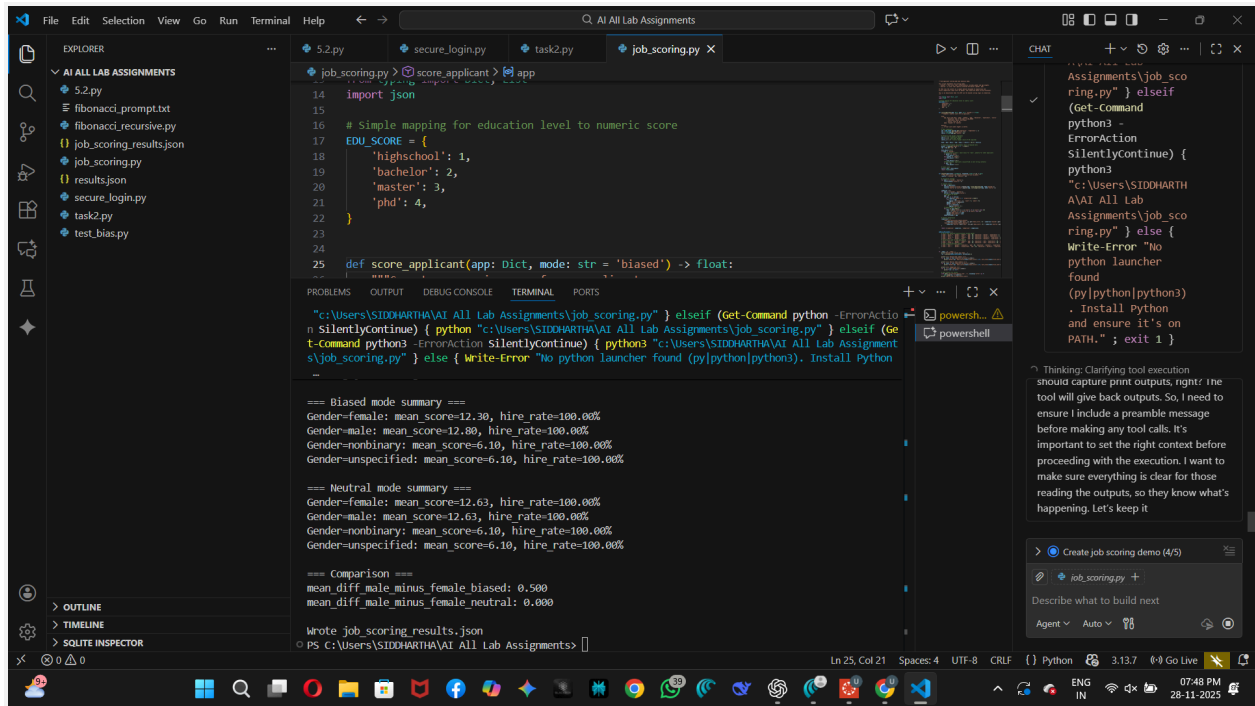
➤ Create job scoring demo (4/5)

job_scoring.py +

Describe what to build next

Agent Auto

Output:-



Task Description#5 (Inclusiveness)

•Code Snippet

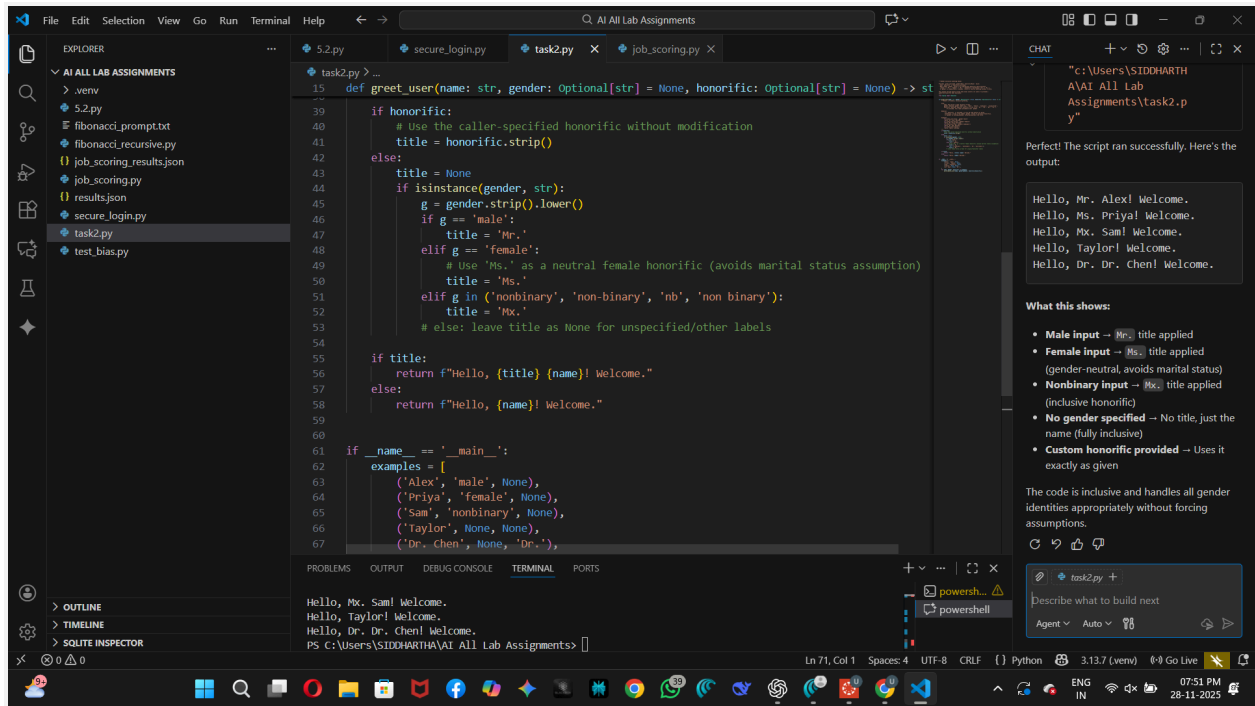
Expected Output

#5•Regenerate code that includes

gender-neutral

als

Input:-



Output:-

